

•



- Project Organization
- [Monitoring](#)
- JMX and Notifications

On this page

JMX and Notifications

Was this page helpful? 

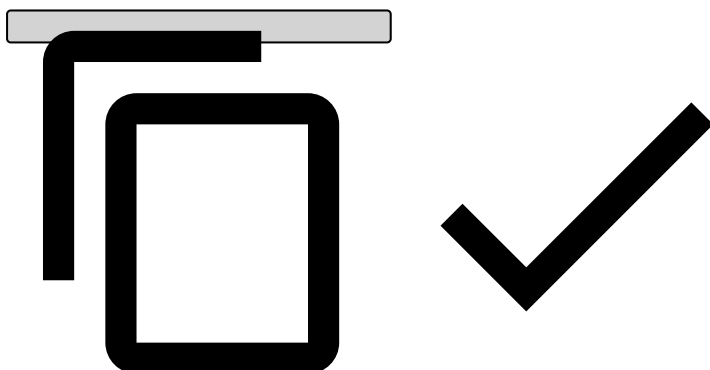
Feedzai Pulse provides a way of notifying monitoring teams when some indicators reach a specified threshold. Moreover, it exposes JMX Beans to extract JVM-related metrics meaning that it can be integrated with third-party monitoring tools such as Nagios.

Notifications

Monitoring teams can configure the system to send notifications when certain thresholds are surpassed. The configuration can be global to all applications running in the server or specific per application and both must be done in **PULSE_HOME/conf/pulse.properties**.

Global configuration

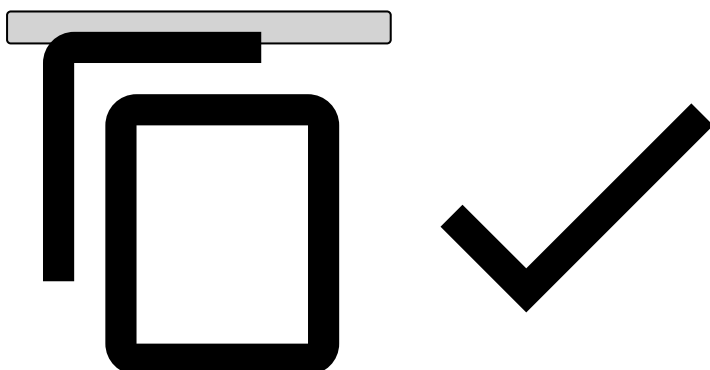
```
pulse.global.appengine.sendemailonfailure=true  
pulse.global.appengine.monitoringemailaddress=[email_protected]  
pulse.global.appengine.heapthreshold=<MEMORY-IN-MB>  
pulse.global.appengine.permgenthreshold=<MEMORY-IN-MB>
```



In order to accomplish the same per application one must change `global` to `app.<APP-NAME>` .

Per-application configuration

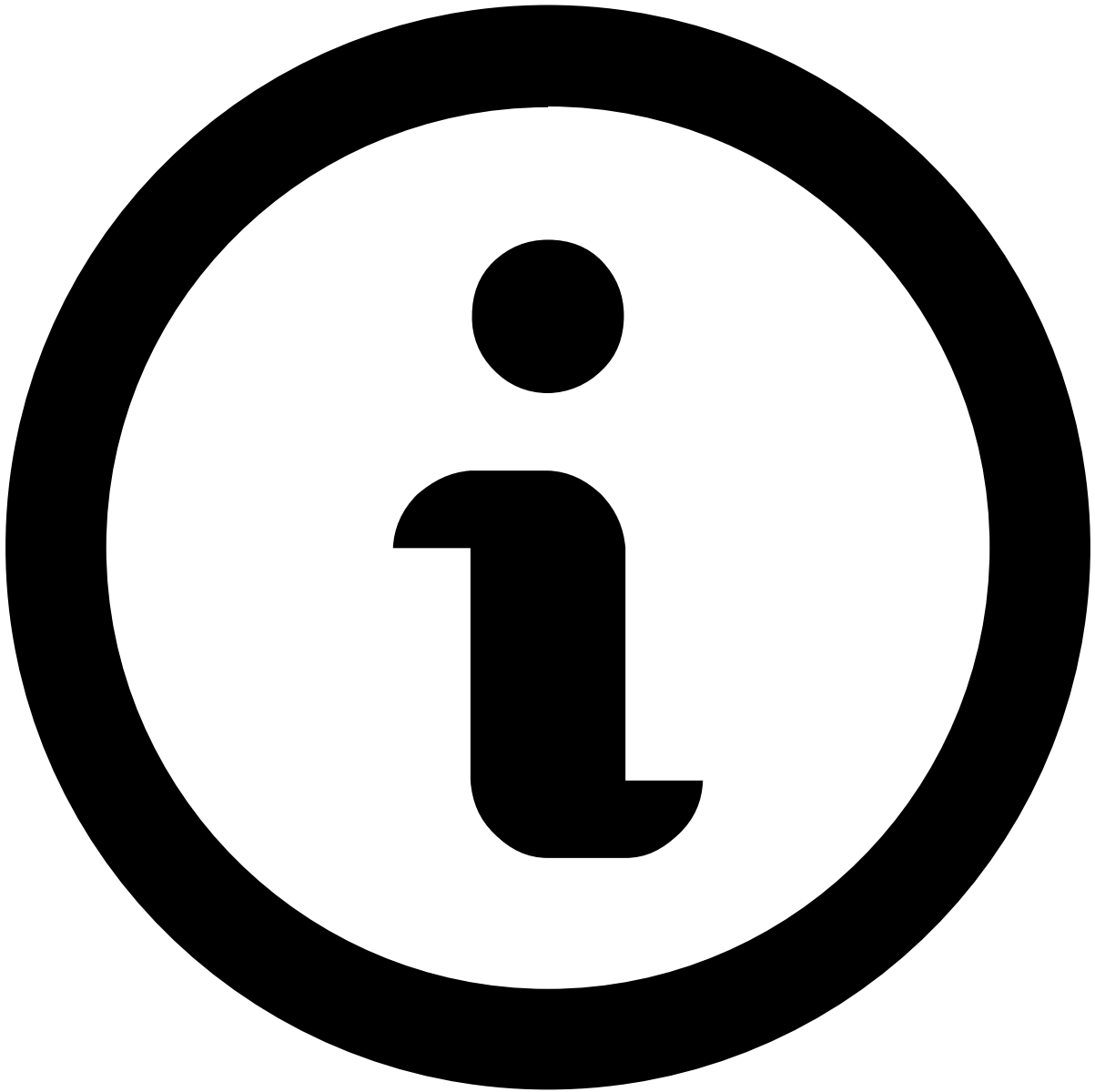
```
pulse.app.<APP-NAME>.appengine.sendemailonfailure=true  
pulse.app.<APP-NAME>.appengine.monitoringemailaddress=[email_protected]  
pulse.app.<APP-NAME>.appengine.heapthreshold=<MEMORY-IN-MB>  
pulse.app.<APP-NAME>.appengine.permgenthreshold=<MEMORY-IN-MB>`
```



The property names are quite self-explanatory so we won't get into much detail with them.

When the amount of memory used reaches the specified threshold, an e-mail is sent to the specified e-mail address informing about the occurrence. If you don't want a particular metric to be monitored, you can delete the entry or assign to it a negative value.

This configuration also notifies when applications crash. That kind of notification cannot be turned off once "sendemailonfailure" is set to "true".



note

Note that e-mail notifications will only be sent if the application crashes while running. If a crash occurs during a publish, or while the application is stopping, a crash notification e-mail is not guaranteed to be sent.

JMX

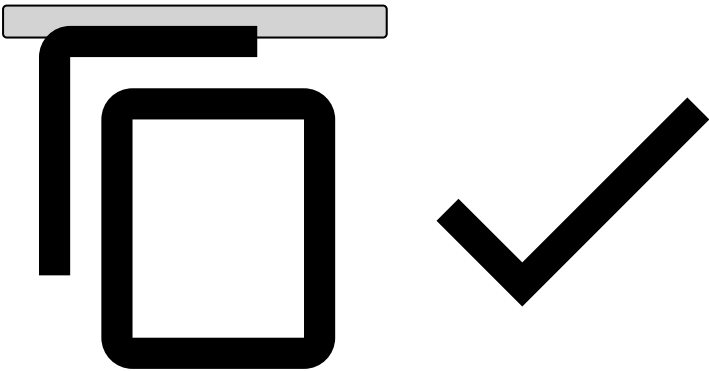
Feedzai Pulse exposes JMX interfaces so that monitoring information can be retrieved and some operations can be performed. The JMX interface is only available when in the same host (localhost). It can be accessed both programmatically or using tools like [IBM Tivoli](#), [Nagios](#).

If a quick and dirty option is needed, even [JConsole](#) can be used to access the JMX endpoints available on Pulse. The following figure shows some of the information available when connected to the Pulse server running a Fraud Prevention application:

Programmatic access

It is possible to access Pulse's JMX endpoints using Java by getting a reference to the `MBean` . In order to accomplish that one must do the following:

```
JMXServiceURL url = new JMXServiceURL("service:jmx:rmi://localhost:9010/jndi/rmi://localhost:9010/jmxrmi");
JMXConnector jmxc = JMXConnectorFactory.connect(url, null);
MBeanServerConnection mbsc = jmxc.getMBeanServerConnection();
ObjectName mbeanName = new ObjectName(StatusConsole.MBEAN_OBJECT_NAME);
StatusConsoleImplMBean console = JMX.newMBeanProxy(mbsc, mbeanName, StatusConsoleImplMBean.class, true);
```



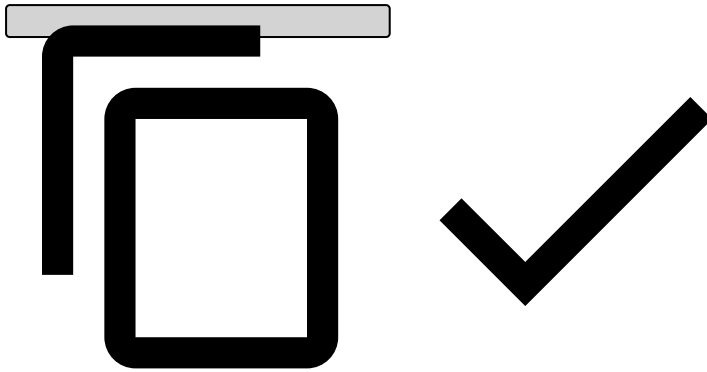
Having the reference, calls to the MBean can be made just by calling the desired methods. The following methods are available:

Method	Return	Description
<code>getName()</code>	String	The console name: StatusConsole
<code>getUptime()</code>	String	A string representation of the uptime
<code>getStartDate()</code>	Date	The start date
<code>getJavaVMVendor()</code>	String	The VM vendor
<code>getJavaVMVersion()</code>	String	The VM version
<code>getOSName()</code>	String	The name of the Operating System
<code>getOSArch()</code>	String	The architecture of the host
<code>getOSVersion()</code>	String	The version of the operating system
<code>getClusterNodes()</code>	List<String>	The list of cluster nodes that compose that Pulse instance. Contains information such as the host, port and whether the instance is hot (the running instance) or not
<code>startShutdown()</code>	Void	Orders Pulse Server to gracefully shutdown itself

The JMX endpoint described above is related to Pulse Server. It's also possible to access information about the running applications. For that, a lookup to the specific AppEngine must be made. This can be done by specifying the data center ID, the application slug (name), the instance IP address and RMI Port as in the following example:

```
JMXServiceURL url = new JMXServiceURL("service:jmx:rmi:///jndi/rmi://localhost:6990/jmxrmi");
JMXConnector jmxc = JMXConnectorFactory.connect(url, null);
MBeanServerConnection mbsc = jmxc.getMBeanServerConnection();
ObjectName mbeanName = new ObjectName(AppStatusConsoleImplMBean.BASE_MBEAN_OBJECT_NAME + "<DATA_CENTER_ID>,nodeType=Apps,app=<APP-SLUG>,node=<HOST>-<PORT>");
```

```
AppStatusConsoleImplMBean console = JMX.newMBeanProxy(mbsc, mbeanName,
AppStatusConsoleImplMBean.class, true);
```



After getting the reference, the following methods will become available:

Method	Return	Description
<code>getName()</code>	String	The name of the console: AppStatusConsole
<code>getUptime()</code>	String	The uptime of the application
<code>getStartDate()</code>	Date	The start date of the application
<code>getAllocatedHeap()</code>	double	The total amount of memory in the JVM (Mb)
<code>getUsedHeap()</code>	double	The amount of used memory in the JVM (Mb)
<code>getInitialHeap()</code>	double	The amount of memory that the JVM initially requests from the OS for memory management (Mb)
<code>getMaxHeap()</code>	double	The maximum amount of memory that can be used for memory management (Mb)
<code>getInitialPermGen()</code>	double	The amount of PermGen that the JVM initially requests from the OS for memory management (Mb)
<code>getMaxPermGen()</code>	double	The maximum amount of PermGen that can be used for memory management (Mb)
<code>getUsedPermGen()</code>	double	The amount of PermGen used (Mb)
<code>getTotalCPU()</code>	double	The CPU usage of all running processes, as a percentage of all processors.
<code>getProcessLoad()</code>	double	The CPU usage of the process running this App Engine, as a percentage of all processors.
<code>getActiveServices()</code>	List<String>	The list of service names running in the application

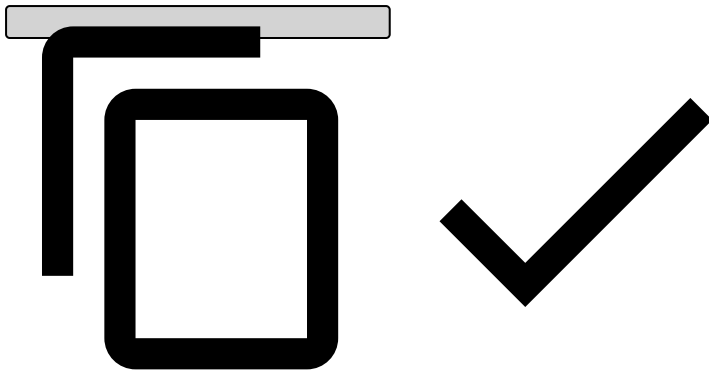
For a reference with the difference between "allocated", "used", "initial" and "max" Heap and PermGen memory spaces, refer to the following [StackOverflow answer](#).

Configuring the JMX Server [🔗](#)

It's sometimes required to configure Pulse's JMX server due to firewall restrictions.

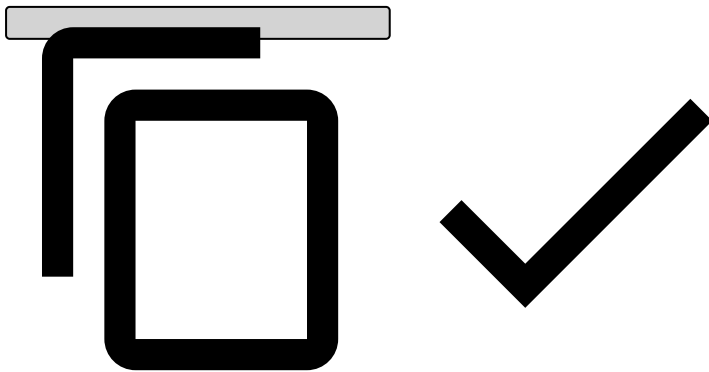
By default Pulse uses port 9010 for JMX access, this can be configured in Pulse's properties:

```
pulse.manager.jmxport=9010
```



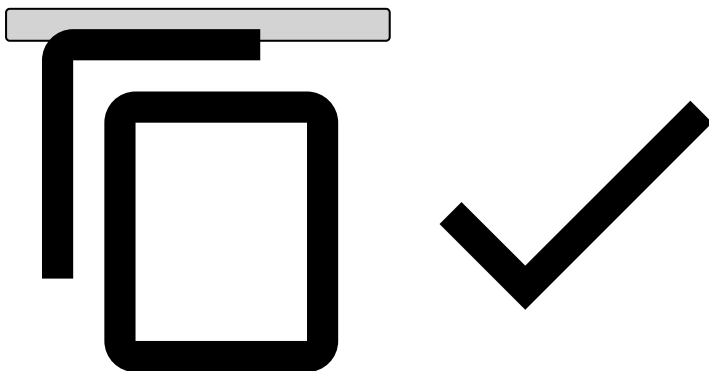
After configuring the port you can use the following JMX URL to connect to:

```
service:jmx:rmi:///<listenAddress>:<jmxport>/jndi/rmi:///<listenAddress>:<jmxport>/jmxrmi
```



As for Application Engines, the default port used to establish the connection is the same as the RMI port. However, a random port will be automatically generated for the actual JMX connection. If you want to use a fixed port you must configure it:

```
#Global setting for all applications  
pulse.global.appengine.jmxport=9011  
  
#Configuration for a specific application  
pulse.app.<app_slug>.appengine.jmxport=9013
```



In this case the JMX URL will be:

```
service:jmx:rmi:///<listenAddress>:<jmxport>/jndi/rmi:///<listenAddress>:<jmxport>/jmxrmi
```

